DDES9991

Unreal Engine Setup for Tethered VR Projects

Based on the Capstone ArchVis VR showcase using UE5.4.4,
with some experimental features tested in UE5.7.1

———

Name LUN LEUNG Christopher
zID 5531230
2026 Term 1

Outline

Getting Started

- Project Setup/Settings/Scalability/Packaging
- Post Process Volume
- Blueprints
- CVars
- Useful Windows/Plugins

VR Specifics

- General Project Considerations
- VR Inputs/Interactions
- VR Pawn
- VR Glitches

Unreal Engine's Fancy Tools

- Nanite
- Lumen
- MegaLights
- Substrate Materials
- MetaHumans

Miscellaneous

- Meta Asynchronous Spacewarp (ASW)
- Nvidia DLSS

Index

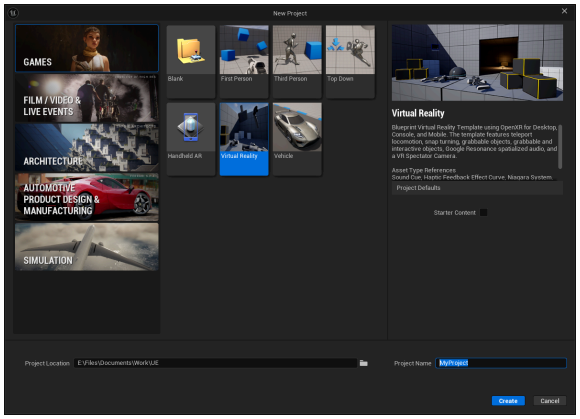
- Useful CVars
- Consolidated Links

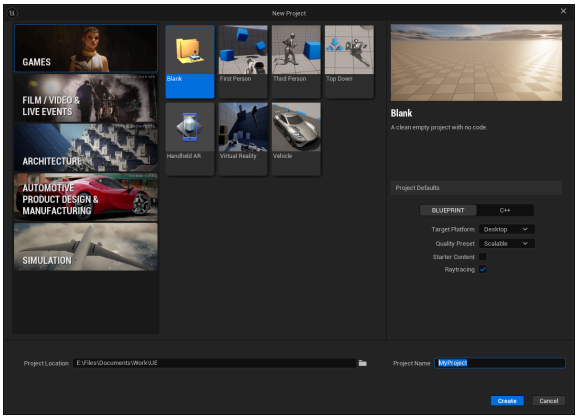
Getting Started

If coming from Unity, Unreal has a great documentation that will be helpful to translate and understand the basics.

—

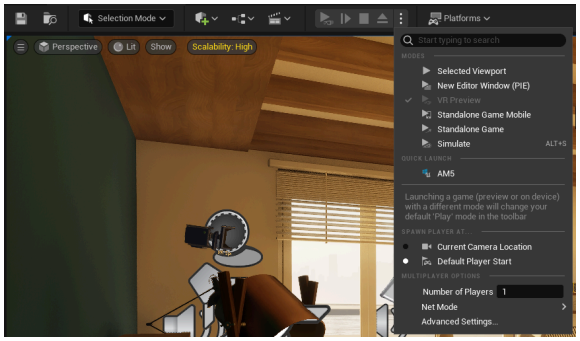
Project Setup





When creating a new VR project, using Unreal's VR template is recommended. Else, you can start with a Blank project with the Target Platform set to 'Desktop' and Quality Preset to 'Scalable'.
If you know how to code in C++, selecting C++ instead of Blueprints might give you more flexibility. This guide uses Unreal's Blueprints system, but most settings should follow a similar path.

NOTE: It is best to launch Quest Link (or your OpenXR Runtime software of choice) if doing anything in VR **before** you open your UE project as you might not be able to access the VR Preview (Play mode) if it's not running beforehand.



Once in the template, you can quickly check if everything is working by navigating to the Play options (click on the 3 dots) then 'VR Preview'. If it is greyed out (screenshot), you might need to connect a headset.

Project Settings

If not using Lumen (baking lights), the default VR Template settings should already be set to the best settings to use for VR. However, additional level/scene assets will be needed to get the best baked results such as Light Mass Importance Volumes and Light Portals.

Here are some helpful links to get started:
Light Baking and Light Maps Guide
Unreal Engine Light Optimisation Tutorial

If planning on using Lumen, below is a list of Project Settings to check:
Platforms / Windows

- Target RHIs -> DirectX12
- D3D12 Targeted Shader Formats -> SM6 Check

Engine / Rendering

Global Illumination & Reflections

- Global Illumination & Reflection Method -> Lumen
- Reflection Capture Resolution -> 64 - 128 if vRAM = 12+GB
- Use Hardware Ray Tracing when available -> Check

Ray Tracing

- Support Hardware Raytracing -> Check
- Ray Traced Shadows -> Check (depending on performance)
- Generate Mesh Distance Fields -> Check

Misc Lighting

- Allow Static Lighting -> Disable

Forward Renderer

- Forward Shading -> Disable

VR - Experiment

- Instanced Stereo -> Enable

Default Settings

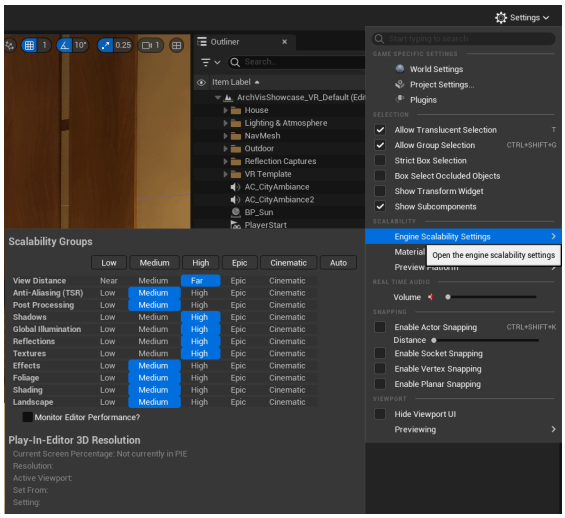
- Anti-Aliasing Method -> TSR
- MSAA Sample Count -> No MSAA
- Support Sky Atmosphere Affecting Height Fog -> Check

It is recommended to turn off 'Allow Static Light' when using Lumen as it saves some static lighting overhead with shader permutations. It also allows Material Ambient Occlusion to work with Lumen Global Illumination.

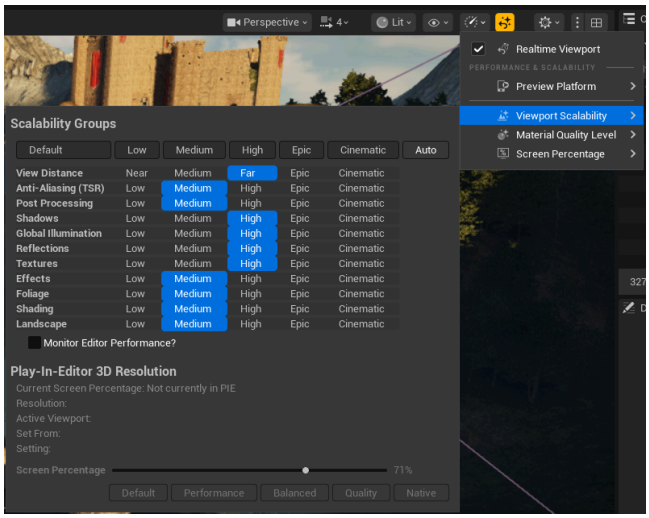
NOTE: Lumen is currently not compatible with the Forward Renderer and quite expensive, check if you have the right hardware and plugins (for e.g. DLSS) to provide a smooth experience.

Project Scalability

UE5.4.4



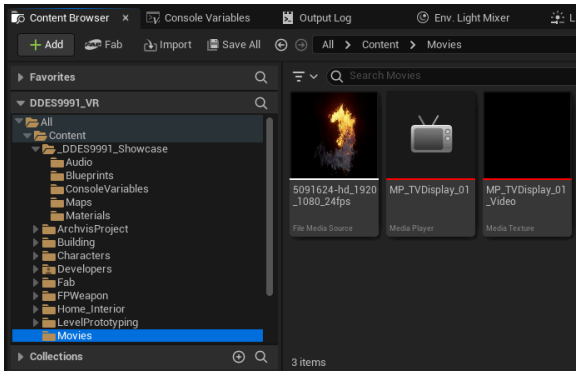
UE5.7.1



For VR, setting the right project scalability will drastically affect performance. The ‘Medium’ preset seems to be a good starting point with GI and Reflections set to ‘High’ if using Lumen (Lumen does not work on Low or Medium scalability). I personally also set the Shadows and Textures to ‘High’ and View Distance to ‘Far’, but results may vary depending on your hardware and level. If not using Lumen, ‘High’ or ‘Epic’ presets might be possible.

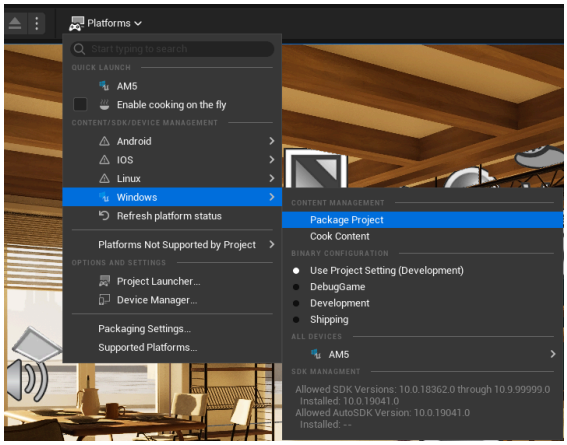
Project Packaging

When packaging/building a project in UE, make sure that only the levels you want to include are set/listed in ‘Project Settings/Packaging/Advanced/List of maps to include in a package build’, and that the Default Map under ‘Project Settings/Maps & Modes/Default Maps/Game Default Map’ is set to one of the listed levels. Else, UE will package all levels present in the project, taking a lot longer.

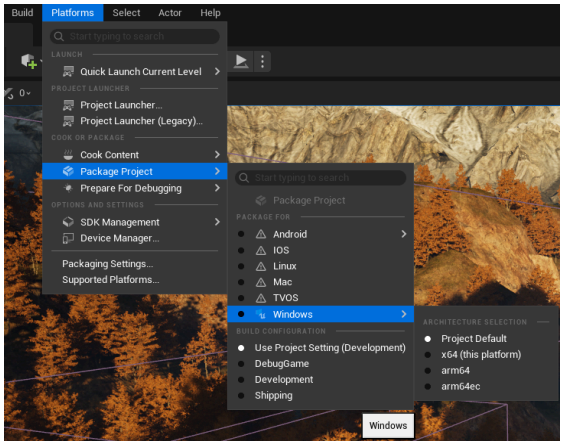


NOTE: Videos should be in the ‘Movies’ folder located at the root of the Content folder (Content/Movies/*) to be included in a Package (following a similar convention to the Streaming Assets folder in Unity).

UE5.4.4



UE5.7.1



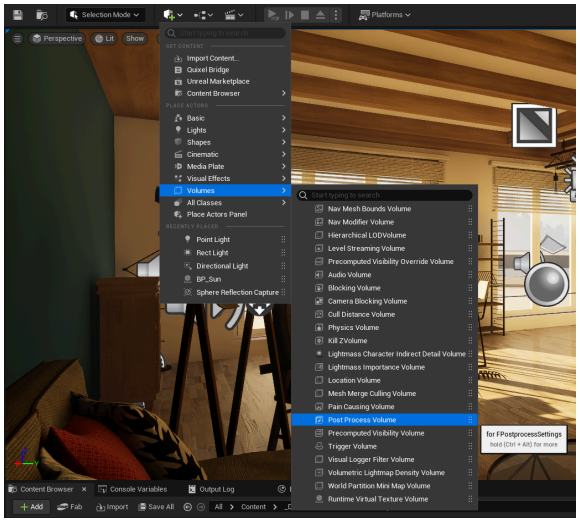
To package the project, go to ‘Platforms/Windows/Package Project’. Make sure to check the ‘Binary Configuration’ based on the purpose of the package (on ‘Development’ by default).

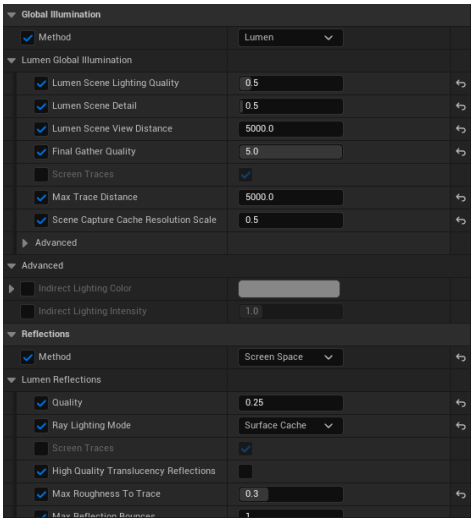
Packaging for standalone VR in UE is complex, requiring multiple external downloads and version checks. Some useful links can be found below to package for the Meta Quest 3:
Meta Quest VR Guide
Packaging for VR in UE Tutorial
Android Studio Download Archive
JDK Download Archive

—

Post Process Volume

A Post Process Volume (PPV) is one of the first items you want to add to your scene. It allows you to add ‘regular’ Post Processing Effects, but also quickly override some Rendering Settings found in your Project Settings such as Lumen.





A few recommended settings to change in the PPV can be found below:

Settings

- Infinite Extents (Unbound) -> Check

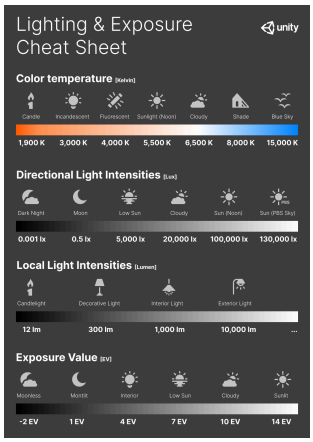
Bloom

Method -> Convolution looks better specially in VR, else Standard (based on hardware)
Intensity -> 0.5-0.8

Exposure

- Metering Mode -> Auto Exposure Histogram, if editing Manual
- Exposure Compensation -1.0 - 0.5, if on Manual 10.0 or adjust accordingly
- Min EV100 -2.0
- Max EV100 2.0
- Speed Up/Down 6.0

NOTE: I would usually recommend using the cheatsheet below when setting up Exposure and Light values in your scene. However, using those values as reference was generating more noise with Lumen (results may vary depending on your hardware and level assets).



Lumen GI

- Lumen Scene Lighting Quality 0.25-0.5
- Lumen Scene Detail 0.25-0.5
- Lumen Scene View Distance 1000-8000
- Final Gather Quality 1.0 - 10.0 (based on hardware)
- Max Trace Distance 1000-8000
- Scene Capture Cache Resolution 0.5

Reflections

- Quality 0.25-0.5
- Ray Lighting Mode -> Surface Cache
- High Quality Translucency -> Uncheck (based on hardware)
- Max Reflection Bounces 2

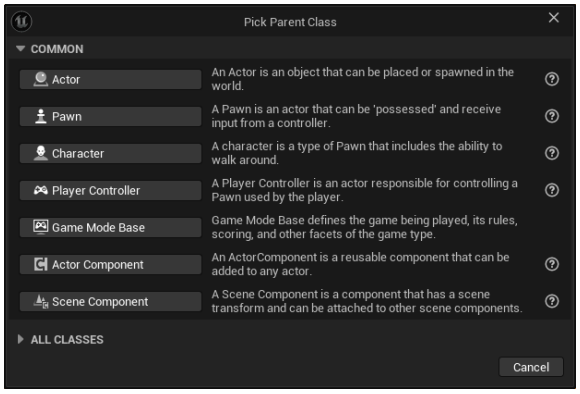
Additional

- Motion Blur Amount 0
- Lens Flare Intensity 0

—

Blueprints

Blueprints are the Visual Scripting system used by Unreal Engine and have a learning curve to get used to. But once you get familiar with most common nodes, you can start building simple to intricate systems. Blueprints have different Classes as seen below, allowing them to perform different tasks based on your needs.



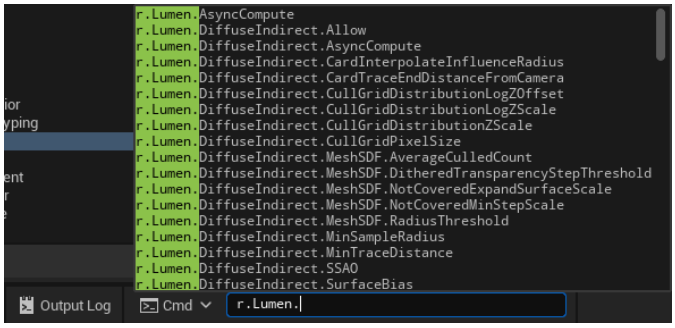
Some helpful links:
Blueprints Documentation
Blueprints for Beginners Tutorial
All Essential UE Nodes Tutorial

NOTE: If you know how to code in C++, it might be easier to build the same systems directly in code, and could additionally open up some restrictions imposed by the Blueprint system.

—

CVars

Console Variables or CVars are an integral part of developing in UE. They control engine behaviour/settings and debugging features. They can be accessed/changed in editor or play mode through the Console (`) or through the configuration .ini files under '[YourProject]/Config/*'

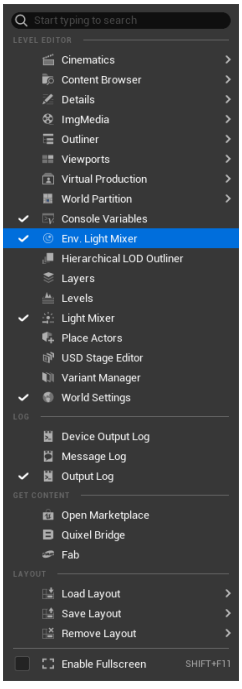


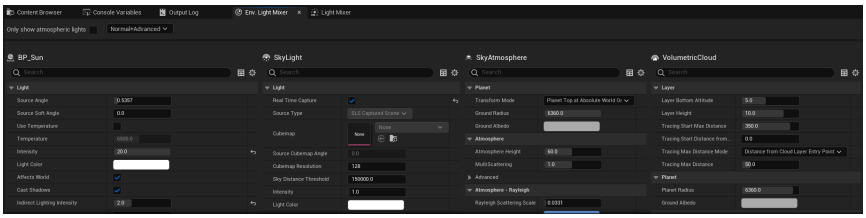
Some useful links:
Console Commands Reference Documentation
Console Variables in C++ Documentation
Unreal Directive Searchable List of CVars Link

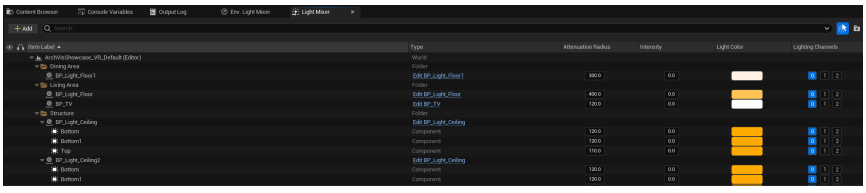
—

Useful Windows/Plugins

If you are coming from Unity, docking the Content Browser (Ctrl + Space) might make your workspace look more familiar.



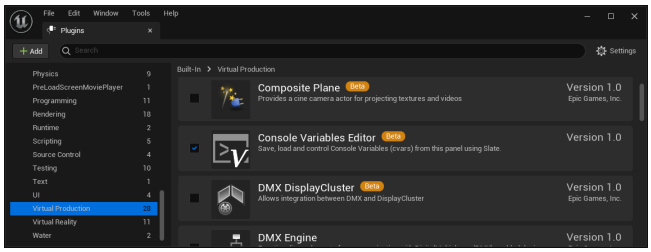


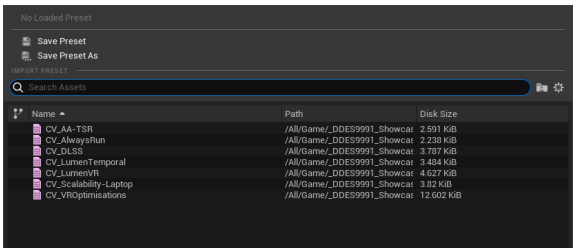


A couple of useful windows are:

1. The Environment Light Mixer, allowing quick access to create and modify your Directional Light, Sky Light, Sky Atmosphere and Volumetric Cloud.
2. The Light Mixer window allows you to access and modify all 'regular' lights in your scene.

A useful plugin to install is the Console Variables Editor. It allows you to save a list of CVars as presets to easily run at any time. A link to the documentation can be found here.





Some extras:

- The Datasmith plugin allows you to import models from a wide range of 3D software seamlessly into UE. Datasmith Exporter Documentation
- Dash is also another plugin allowing you to easily create procedural assets in UE that blend seamlessly. Dash for UE link

VR Specifics

—

General Project Considerations

Since this guide focuses on tethered VR, knowing the hardware to which the VR device will be tethered, will affect the possible experience. Unreal Engine relies heavily on vRAM (GPU), and as of early 2026, a minimum of 12GB of vRAM is recommended for Lumen to work properly. If not using Lumen, 8GB of vRAM should be enough.
Other recommendations:

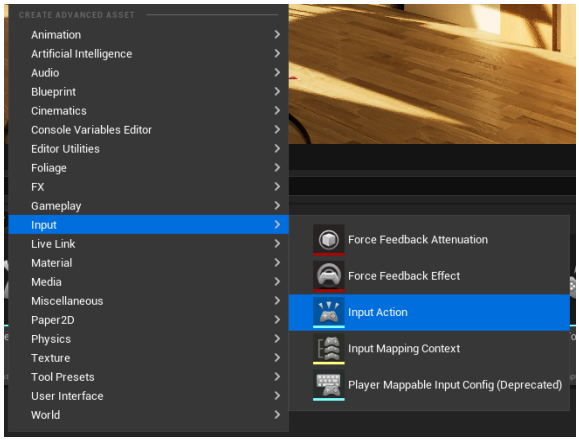
- RAM 32GB or more
- CPU Quad-core 2.5GHz
- SSD with 20+GB of storage space available (for reference, the current ArchVis project is about 15GB in size and 1.8GB for a Package/Build).

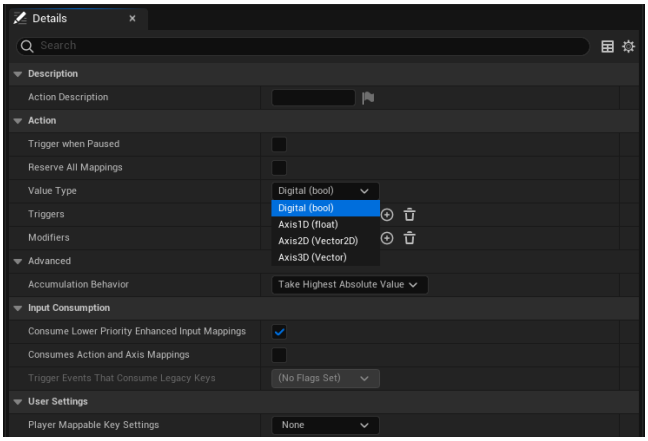
Some useful links for getting started in UE VR:
Best XR Practices in UE Documentation
Virtual Reality and OpenXR in UE5 Unreal Fest
Utilising Lumen in High Fidelity VR Inside Unreal

—

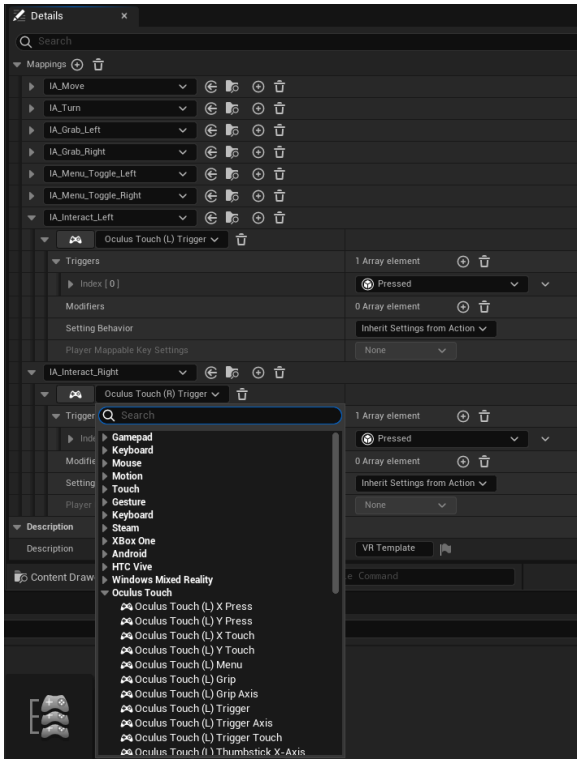
VR Input/Interactions

To create custom inputs, you first need to create a custom Input Action (Right Click in your Content Browser -> Input -> Input Action). From there you can set the action as a Bool, Float, Vector2 (joystick) or Vector3 (motion controller).

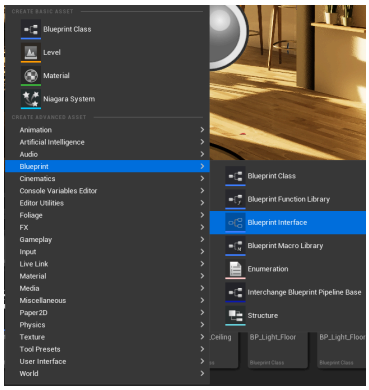


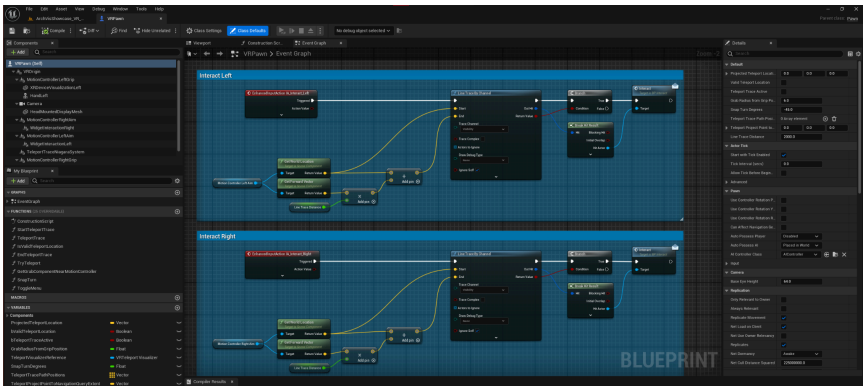


Next, add the Input Action(s) (IA) to the Input Mapping Context (IMC_Default in the VR Template) and set them to the desired Controller Buttons - Decide when the IA(s) should be triggered: on Press, Release, Hold, etc. (Unreal has some popular VR controller input mappings included when using the VR Template)

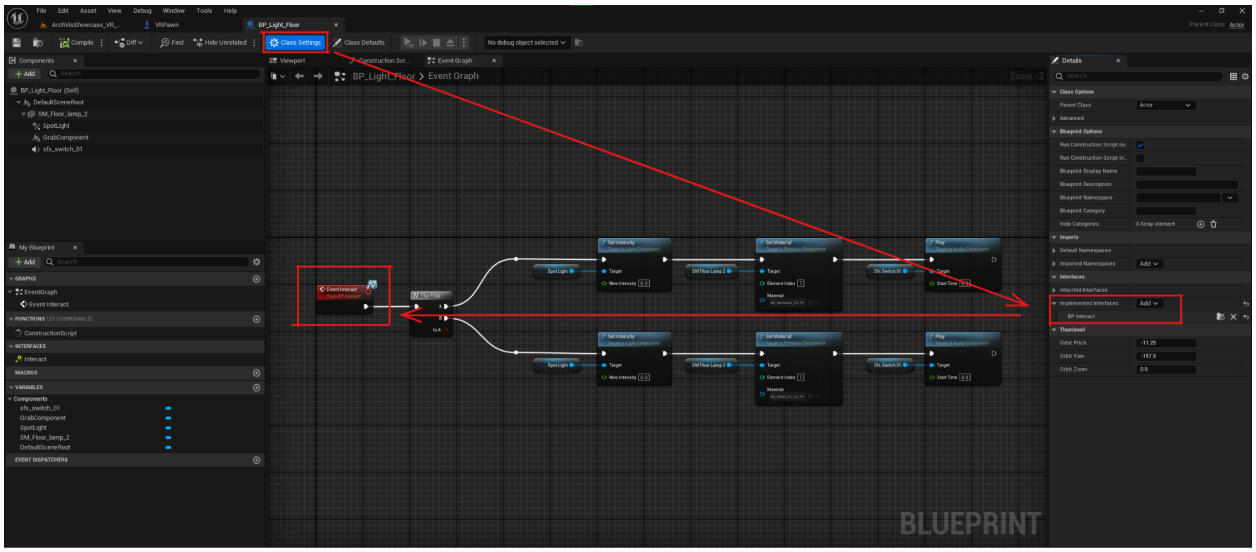


Now create a Blueprint Interface (Right Click in your Content Browser -> Blueprint -> Blueprint Interface) that will be used in the VR Pawn/Player Controller Blueprint, to tie it to the custom Input Action(s), and take into account its functions/calculations (for e.g. where the controller pointing).





That Blueprint Interface can then be used in any Blueprint Actor(s) to perform a function by adding it as an Implemented Interface under the Class Settings (steps below).



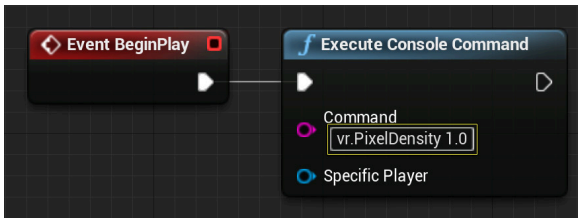
—

VR Pawn

Apart from adding the custom Blueprint Interface and Custom IAs to the VR Pawn, 2 other important components can be modified to optimise the VR experience.

vr.PixelDensity

The Executable Console Command node holding the vr.PixelDensity value is a CVar that can be used to under/over-sample the resolution of your HMD, significantly affecting performance. I would recommend any value higher than 0.7 if running on the Meta Quest 3.

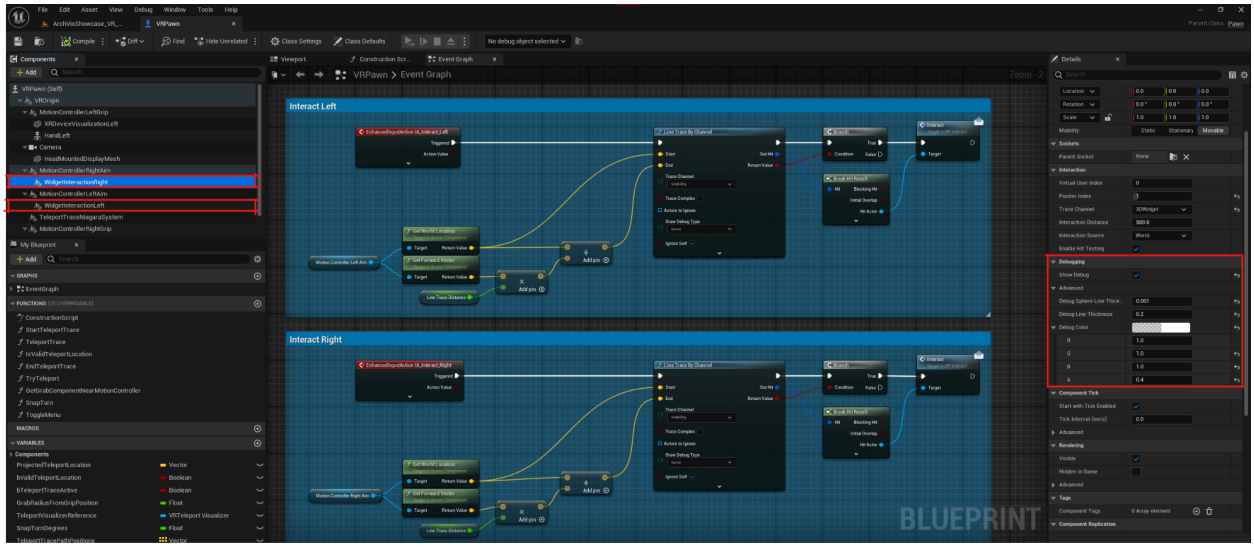


More info can be found here.

Debugging Lines

By default, the hand/controller meshes do not have any shooting rays that you might be familiar with (when using Meta's Interaction Toolkit), making it difficult to interact with distant objects.

You can obtain a similar function by enabling the 'Debugging' features inside the Widget Interaction (found under each Motion Controller Aim), and changing their thickness, transparency and colour values.



—

VR Glitches

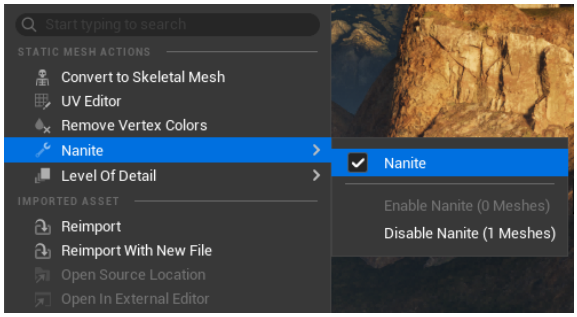
Virtual Shadow Maps seem to not work properly in UE5.4.4. You can fix them by using the following CVar ‘r.Shadow.Virtual.OnePassProjection 0’

Unreal Engine’s Fancy Tools

—

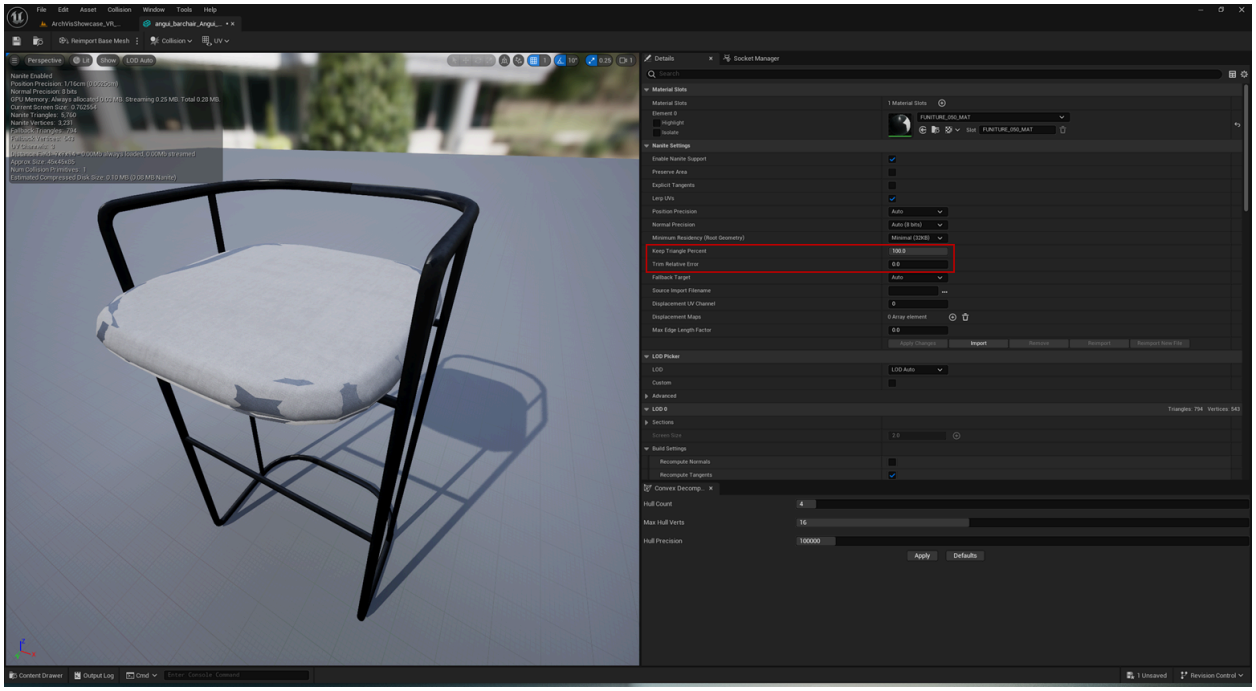
Nanite

Nanite is pretty straightforward and can be applied to any static mesh in UE. It can be created on import or later on enabled and applied to individual meshes. It allows for pixel scale detail, high object count and automatic handling of LODs. Nanite Documentation

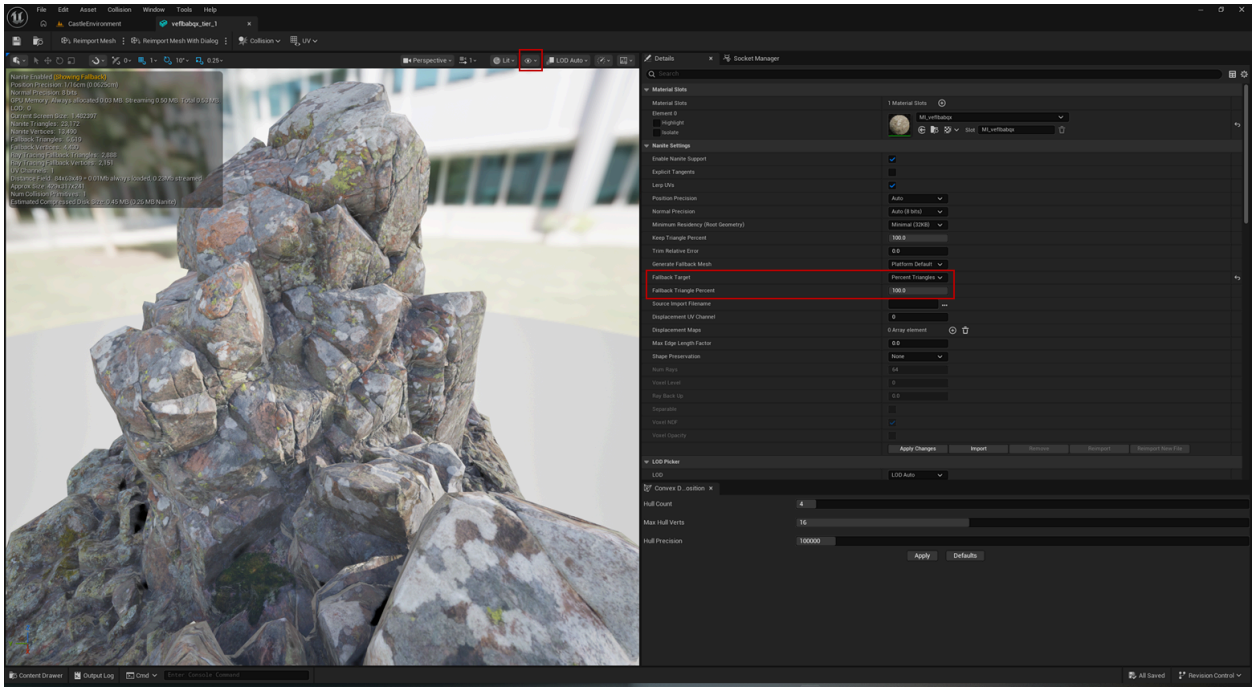


Keep in mind that Nanite does have an upfront cost, and is best used on high poly count objects or if there are multiple instances of the object in the scene.

Some overlapping issues can happen with Nanite in areas where the meshes are thin or rounded off. However, they can be easily fixed by adjusting the Relative Error values.



A Fallback mesh is created in parallel to the Nanite mesh, used for lighting/shadow casting. If your Nanite mesh looks fine, but has shadow artefacts, it might be due to the Fallback mesh being too far off the Nanite mesh. You can see the Fallback mesh by going to ‘Show -> Fallback’, and adjust the Fallback Target and Fallback Relative Error/TrianglePercent values.



—

Lumen

Lumen is one of the most expensive tools in UE and its use-case should be decided based on the hardware available. The biggest issue with Lumen is the amount of noise present in the Shadows and Reflections. The easiest fix is to adjust the ‘quality’ values using the Post Process Volume (Lighting/Reflection Quality, Scene Detail, Final Gather). However, since we are constrained to hardware, those values have a limit where going beyond them will hinder performance.

NOTE: Emissive materials propagate through Lumen at no additional performance cost, but there’s a limit to how small and bright they can be (further adding noise if too small or bright).

A work around to reduce Lumen noise is by using some of the CVars listed below. However, these might have different effects depending on the running hardware.

Some additional useful links:
Global Illumination Documentation
Lighting workflow in UE Guide
Lighting in UE5 Tutorial

Eliminate Noise in Lumen Reflections Link 1, Link 2
Using Lumen with TAA Guide
Lumen Performance Guide

r.Lumen.ScreenProbeGather.DownsampleFactor 12-18
r.Lumen.ScreenProbeGather.TracingOctahedronResolution 16
r.Lumen.ScreenProbeGather.FullResolutionJitterWidth 0.5
r.Lumen.ScreenProbeGather.RadianceCache.ProbeResolution 16

r.LumenScene.Radiosity.ProbeSpacing 8
r.LumenScene.Radiosity.HemisphereProbeResolution 3
r.Lumen.Reflections.ScreenSpaceReconstruction.TonemapStrength 1
r.Lumen.Reflections.MaxRoughnessToTrace 0.2-0.4

r.RayTracing.Culling 3
r.RayTracing.Culling.Radius 15000
r.RayTracing.Culling.Angle 0.5

—

MegaLights

MegaLights is only available in UE5.6 and up and seems to be best used when working in tandem with Lumen. It allows for “orders of magnitude” more dynamic and shadow area lights in a scene, by not only reducing the cost of dynamic shadowing but also the cost of the shading itself.

To use MegaLights, it should first be activated in the Project Settings. It can subsequently be overwritten in the Post Process Volume. Megalights does not work on a per-light basis and is dependent on Ray Tracing to cast shadows, thus imposing a hardware restriction.

Some useful links:
Megalights Documentation
Megalights Tutorial

—

Substrate Materials

Substrate materials use Unreal’s ‘slab’ approach to authoring materials, replacing fixed shading models. They can be useful in detailed product visualisation where accurate representation of complex surfaces are needed (advance layering and blending).

The ‘slab’ is comprised of 2 parts:

1. The interface: surface boundary - Roughness, Normal, Diffuse Albedo, F0 and F90

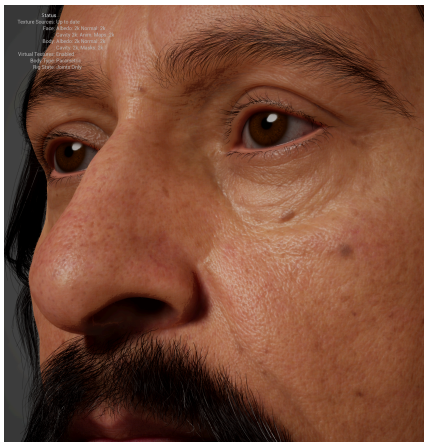
2. The medium: volume under surface where light is scattered and absorbed - MFP

Some useful links:
Substrate materials Documentation
Introduction to Substrate Materials Video
Substrate Beginner's Guide Tutorial

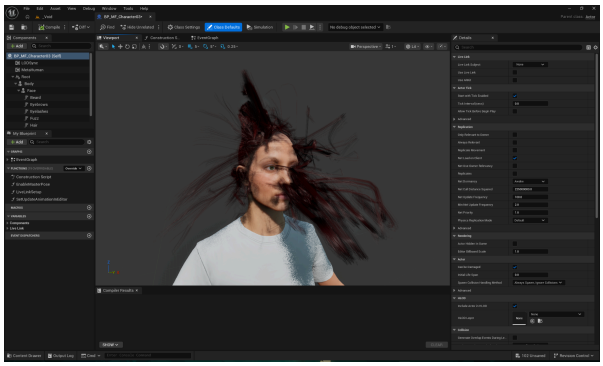
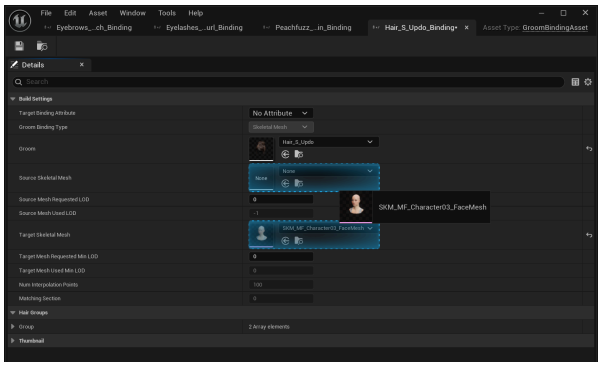
—

MetaHumans

MetaHumans allows for the creation of ultra-realistic looking characters that could be used for virtual production/animation. Creating, editing and assembling the characters themselves require top of the line hardware (about 16GB of vRAM or more). I was only able to assemble the pre-made characters with the hardware available.

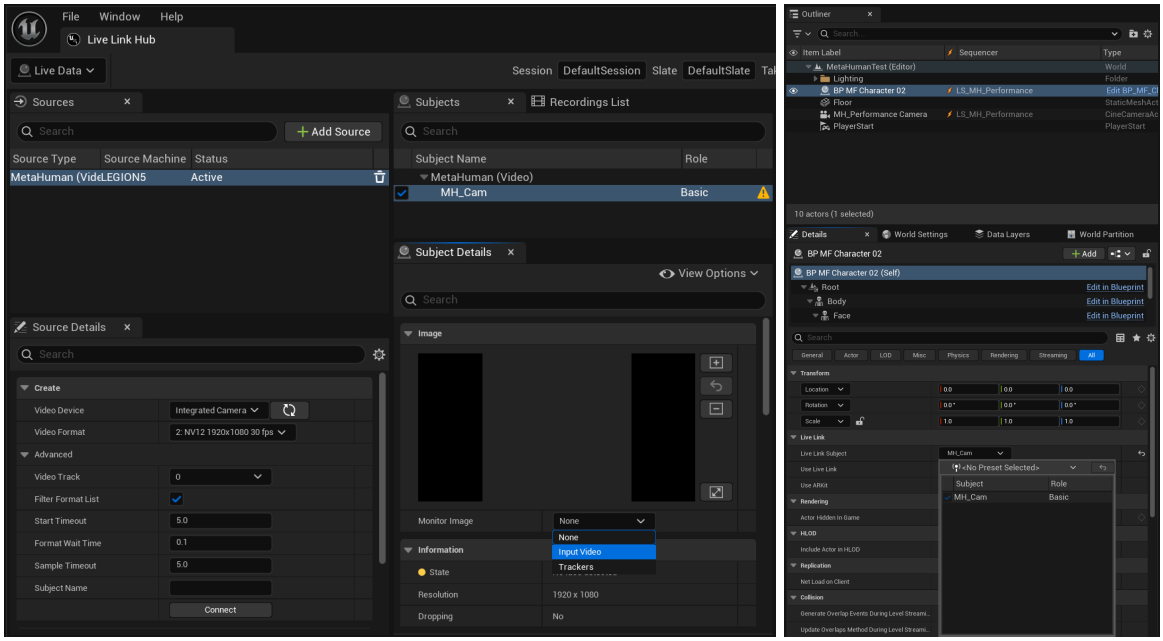



I ran into an issue after assembling my character, where the Skeletal Face Mesh had to be relinked on every 'Groom Binding' asset.

Once properly assembled, you can use the created MetaHuman blueprint for live facial tracking or lip syncing.

For Live Facial Tracking you will need to enable the Live Link, Live Link Control Rig and Live Link Hub plugins. Once enabled you can launch Live Link by going to Tools -> Live Link Hub. MetaHumans now works with any camera, but having one with depth sensors (LIDAR) might give better results.



In Live Link Hub you'll have to set your camera by:

1. Checking if its set to 'Live Data' in the top Left
2. Adding a Source -> MetaHuman (Video)
3. Selecting your camera/webcam
4. Add a name to the camera (e.g. MH_Cam)
5. Connect

You can check if everything is working fine by switching the Monitor Image to 'Input Video'

Now go back to your UE project and drag the MetaHuman blueprint into the scene. Select the character, under the Live Link -> Live Link Subject you should now see your camera. Select it and check 'Use Live Link' to start tracking.

NOTE: If you don't see your camera under the Live Link Subjects, you might need to change the Unicast Endpoint address. Go to Project Settings -> UDP Messaging (usually at the very bottom) -> Unicast Endpoint.

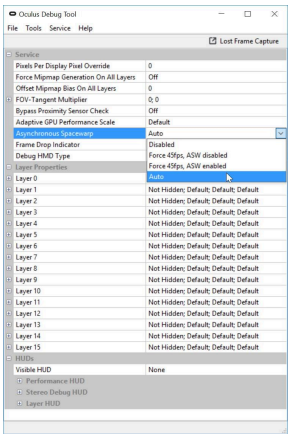
Useful links:
Webcam LiveLink for Metahumans Tutorial
MetaHumans Tutorial
Live Link Troubleshooting

Miscellaneous

—

Meta's Asynchronous Spacewarp

A technical limitation that I found when using the Meta Quest 3 with Quest Link, is Meta's Asynchronous Spacewarp (ASW) option that is activated by default. ASW in short tries to save on resources by limiting the FPS of the VR device to half of its refresh rate, and applies a framerate smoothing technique to produce nearly the same output (documentation here).



However, I found some artefacts around the edges of the frame when it was being activated in UE and decided to fully disable it through the Oculus Debugging Tool (usually found at C:\Program Files\Oculus\Support\oculus-diagnostics\OculusDebugTool)

Note that this setting resets to Auto every time the Quest Link app is restarted. It can be modified permanently through Regedit:

1. Go to 'HKEY_LOCAL_MACHINE\SOFTWARE\Oculus'
2. Add a new 'DWORD' called "AswDisabled" and set to 1
3. Restart OVRServer.

—

Nvidia DLSS

The DLSS plugin from Nvidia is available for versions of UE5.4 and up. Nvidia's upscaler seems to have better image quality results compared to UE's TSR, but has a bigger upfront cost. Using DLSS also restricts you to Temporal AA, which might create ghosting on transparent materials.

Using DLSS at 58% screen percentage (DLSS Auto) and 1.0 VR Pixel Resolution gave a 20fps boost (from 40 to 60fps) without changing any other setting. Decreasing the Pixel Density to 0.8 made the experience run at 75-80fps with little change in visual quality.





Some useful links:
Nvidia DLSS Documentation & Plugin
DLSS3 Tutorial (Steps to install should still be the same)
Install DLSS4 in Unreal Engine 5 Tutorial

The plugins download file includes a sample scene where DLSS features can be tested. If you have the Console Variables Editor plugin, you'll be able to see the CVars changing in realtime.

CVars
r.NGX.DLSS.Enable 1 (Enabling DLSS will override other AAs)
r.ScreenPercentage 100.0 for DLAA, 66.7 for Quality, 58.0 for Balanced, 50.0 for Performance.
r.Streamline.DLSSG.Enable 2 (Enables Nvidia Frame Generation)
t.Streamline.Reflex.Enable 1 (Enables Nvidia Reflex to reduce latency)
r.NIS.Sharpness 0.2 (Adds some sharpness to the final image)
r.NGX.DLSS.DenoiserMode 0
r.ShadowDenoiser 2

Index

—

Useful CVars

These can be inserted in the DefaultEngine.ini file found in the '[Your Project Name]/Config' folder to run by default or be included in your package.

General

r.Streaming.PoolSize 0
r.Streaming.LimitPoolSizeToVRAM 1

VR & Fixes

vr.PixelDensity 0.8-1.0
r.Shadow.Virtual.OnePassProjection 0

Lumen

r.LumenScene.Radiosity.Temporal.MaxFramesAccumulated (Default 4, Range between 4-32)
r.LumenScene.Radiosity.HemisphereProbeResolution (Default 4, Range between 4-16 vRAM intensive)

r.Lumen.Reflections.ScreenSpaceReconstruction.TonemapStrength 1
r.Lumen.Reflections.MaxRoughnessToTrace 0.2-0.4

r.Lumen.ScreenProbeGather.DownsampleFactor 12-18
r.Lumen.ScreenProbeGather.TracingOctahedronResolution 16
r.Lumen.ScreenProbeGather.FullResolutionJitterWidth 0.5
r.Lumen.ScreenProbeGather.RadianceCache.ProbeResolution 16
r.Lumen.ScreenProbeGather.IrradianceFormat 1
r.Lumen.ScreenProbeGather.StochasticInterpolation 1
r.Lumen.ScreenProbeGather.ScreenTraces.HZBTraversal.FullResDepth 0
r.Lumen.ScreenProbeGather.Temporal 1
r.Lumen.ScreenProbeGather.Temporal.ClearHistoryEveryFrame 0
r.Lumen.ScreenProbeGather.Temporal.DistanceThreshold 0.01
r.Lumen.ScreenProbeGather.Temporal.FastUpdateModeUseNeighborhoodClamp 1
r.Lumen.ScreenProbeGather.Temporal.MaxFastUpdateModeAmount 0.9
r.Lumen.ScreenProbeGather.Temporal.MaxFramesAccumulated 16.0
r.Lumen.ScreenProbeGather.Temporal.MaxRayDirections 8
r.Lumen.ScreenProbeGather.Temporal.NormalThreshold 45
r.Lumen.ScreenProbeGather.Temporal.RejectBasedOnNormal 0

r.Lumen.TranslucencyReflections.FrontLayer.Allow 0

r.Lumen.TranslucencyVolume.GridPixelSize 64
r.Lumen.TranslucencyVolume.TraceFromVolume 0

r.Lumen.TraceMeshSDFs.Allow 0

r.LumenScene.Radiosity.ProbeSpacing 8
r.LumenScene.Radiosity.HemisphereProbeResolution 3

r.RayTracing.Culling 3
r.RayTracing.Culling.Radius 15000
r.RayTracing.Culling.Angle 0.5

DLSS
r.NGX.DLSS.Enable 1 (Enabling DLSS will override other AAs)
r.ScreenPercentage 100.0 for DLAA, 66.7 for Quality, 58.0 for Balanced, 50.0 for Performance.
r.NIS.Sharpness 0.2
r.Streamline.DLSSG.Enable 2 (Enables Nvidia Frame Generation)
t.Streamline.Reflex.Enable 1 (Enables Nvidia Reflex to reduce latency)
r.NGX.DLSS.DenoiserMode 0
r.ShadowDenoiser 2

—

Consolidated Links

Documentation

Unreal Engine for Unity Devs Documentation
Blueprints Documentation
Nanite Documentation
Global Illumination Documentation
Virtual Shadow Maps Documentation
CVars Documentation
Nanite Foliage Documentation
PCG Documentation
Landscape technical guide Documentation
Substrate materials Documentation
Megalights Documentation
Live Link Troubleshooting
Best XR Practices in UE Documentation
Datasmith Exporter Documentation
Meta Asynchronous Spacewarp Documentation

Tutorials/Videos

UE Beginner/General Tutorial
Data layers Tutorial
Megalights Tutorial
Introduction to Substrate Materials Video
Substrate Beginner's Guide Tutorial
Webcam LiveLink for Metahumans Tutorial
MetaHumans Tutorial
Audio Visualiser Tutorial
Packaging for VR in UE Tutorial
Lumen & Nanite in VR Tutorial
VR with Unreal Engine 5 Tutorial
Procedural Grip Poses Tutorial
Optimise Lumen for VR Tutorial
Lighting in UE5 Tutorial
Eliminate Noise in Lumen Reflections Link 1, Link 2
DLSS3 Tutorial
Install DLSS4 in Unreal Engine 5 Tutorial

Written Guides/Other

Lighting workflow in UE Guide
Virtual Reality and OpenXR in UE5 Unreal Fest

A 5000^2 ft VR experience for Four Seasons Lake Austin Unreal Fest
Dash for UE link
Utilising Lumen in High Fidelity VR Inside Unreal
Lighting in UE5 Unreal Fest
Using Lumen with TAA Guide
Lumen Performance Guide
DLSS Plugin